PHOTO PRACTICALS

Answers for the 7 practical questions shown in the supplied photo

No.	Practical
1	Form Validation using JavaScript
2	AngularJS Shopping List
3	Simple Calculator Application using React JS
4	NodeJS Server serving Static HTML and CSS without Express
5	Counter using ReactJS
6	Simple Login Form using React JS
7	Chat Module using HTML, CSS and JavaScript

Practical 1

Form Validation using JavaScript

```
<!DOCTYPE html>
<html>
<head>
<title>Registration Form</title>
<script>
function VALIDATEDETAIL() {
    var name = document.forms["RegForm"]["Name"];
    var email = document.forms["RegForm"]["EMail"];
    var phone = document.forms["RegForm"]["Telephone"];
    var password = document.forms["RegForm"]["Password"];
    var address = document.forms["RegForm"]["Address"];

    if (name.value == "") {
        window.alert("Please enter your name.");
        name.focus();
        return false;
    }

    if (address.value == "") {
        window.alert("Please enter your address.");
        address.focus();
        return false;
    }

    if (email.value == "") {
        window.alert("Please enter a valid e-mail address.");
        email.focus();
        return false;
    }

    if (email.value.indexOf("@") < 0) {
        window.alert("Please enter a valid e-mail address.");
        email.focus();
        return false;
    }

    if (email.value.indexOf(".") < 0) {
        window.alert("Please enter a valid e-mail address.");
        email.focus();
        return false;
    }

    if (phone.value == "") {
        window.alert("Please enter your telephone number.");
        phone.focus();
        return false;
    }

    if (password.value == "") {
        window.alert("Please enter your password.");
        password.focus();
        return false;
    }

    return true;
}
</script>
</head>

<body>
<h1 style="text-align:center">REGISTRATION FORM</h1>

<form name="RegForm" onsubmit="return VALIDATEDETAIL()">
    <p>Name:
        <input type="text" size="65" name="Name">
    </p>

    <p>Address:
        <input type="text" size="65" name="Address">
    </p>

    <p>E-mail Address:
        <input type="text" size="65" name="EMail">
    </p>

    <p>Password:
        <input type="password" size="65" name="Password">
```

```
    </p>

    <p>Telephone:
        <input type="text" size="65" name="Telephone">
    </p>

    <p>
        <input type="submit" value="Send">
        <input type="reset" value="Reset">
    </p>
</form>
</body>
</html>
```

Practical 2

AngularJS Shopping List

```
<!DOCTYPE html>
<html>
<head>
<title>Shopping List</title>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>

<body ng-app="shoppingApp" ng-controller="shoppingCtrl">

<h2>Shopping List</h2>

<input type="text" ng-model="newItem" placeholder="Enter item">
<button ng-click="addItem()">Add Item</button>

<div ng-if="shoppingList.length > 0">
    <h3>Items:</h3>
    <ul>
        <li ng-repeat="item in shoppingList">
            {{item}}
            <button ng-click="removeItem($index)">Remove</button>
        </li>
    </ul>
</div>

<div ng-if="shoppingList.length == 0">
    <p>No items in the list.</p>
</div>

<script>
var app = angular.module("shoppingApp", []);

app.controller("shoppingCtrl", function($scope) {

    $scope.shoppingList = [];

    $scope.addItem = function() {
        if ($scope.newItem && $scope.newItem.trim() != "") {
            $scope.shoppingList.push($scope.newItem.trim());
            $scope.newItem = "";
        }
    };

    $scope.removeItem = function(index) {
        $scope.shoppingList.splice(index, 1);
    };

});
</script>

</body>
</html>
```

Practical 3

Simple Calculator Application using React JS

```
// App.js
import React, { useState } from 'react';
import './App.css';

function App() {
    const [currentInput, setCurrentInput] = useState('');
    const [previousInput, setPreviousInput] = useState('');
    const [operation, setOperation] = useState('');

    const handleNumberClick = (number) => {
        setCurrentInput(currentInput + number);
    };

    const handleOperationClick = (operator) => {
        setOperation(operator);
        setPreviousInput(currentInput);
        setCurrentInput('');
    };

    const handleClear = () => {
        setCurrentInput('');
        setPreviousInput('');
        setOperation('');
    };

    const handleEvaluate = () => {
        if (previousInput && operation && currentInput) {
            const prev = parseFloat(previousInput);
            const current = parseFloat(currentInput);
            let result;

            switch (operation) {
                case '+':
                    result = prev + current;
                    break;
                case '-':
                    result = prev - current;
                    break;
                case '*':
                    result = prev * current;
                    break;
                case '/':
                    result = prev / current;
                    break;
                default:
                    return;
            }

            setCurrentInput(result.toString());
            setPreviousInput('');
            setOperation('');
        }
    };

    return (
        <div className="calculator">
            <div className="display">
                <div>{previousInput} {operation}</div>
                <div>{currentInput}</div>
            </div>

            <div className="buttons">
                <button onClick={() => handleNumberClick('7')}>7</button>
                <button onClick={() => handleNumberClick('8')}>8</button>
                <button onClick={() => handleNumberClick('9')}>9</button>
                <button onClick={() => handleOperationClick('/')}>/</button>

                <button onClick={() => handleNumberClick('4')}>4</button>
                <button onClick={() => handleNumberClick('5')}>5</button>
                <button onClick={() => handleNumberClick('6')}>6</button>
                <button onClick={() => handleOperationClick('*')}>*</button>

                <button onClick={() => handleNumberClick('1')}>1</button>
                <button onClick={() => handleNumberClick('2')}>2</button>
                <button onClick={() => handleNumberClick('3')}>3</button>
                <button onClick={() => handleOperationClick('-')}>-</button>
                <button onClick={() => handleNumberClick('0')}>0</button>
```

```
                <button onClick={handleClear}>C</button>
                <button onClick={handleEvaluate}>=</button>
                <button onClick={() => handleOperationClick('+')}>+</button>
            </div>
        </div>
    );
}

export default App;

// App.css
.calculator {
    width: 260px;
    margin: 100px auto;
    background-color: #f4f4f4;
    padding: 20px;
    border-radius: 10px;
}

.display {
    text-align: right;
    margin-bottom: 20px;
}

.buttons button {
    width: 50px;
    height: 50px;
    margin: 5px;
    font-size: 18px;
}
```

Practical 4

NodeJS Server serving Static HTML and CSS without Express

```
const http = require('http');
const url = require('url');
const fs = require('fs');
const path = require('path');

const server = {};

const baseDir = path.join(__dirname, '../');

const mimeTypes = {
    '.html': 'text/html',
    '.jpg': 'image/jpeg',
    '.css': 'text/css',
    '.js': 'text/javascript',
    '.png': 'image/png',
    '.ico': 'image/x-icon'
};

const httpServer = http.createServer((request, response) => {

    const parsedUrl = url.parse(request.url, true);
    let pathName = parsedUrl.pathname;

    const extensionName = path.extname(pathName);

    if (extensionName === '') {
        pathName = `${pathName}/index.html`;
    }

    const responseContentType = getContentType(pathName);
    response.setHeader('Content-Type', responseContentType);

    fs.readFile(`${baseDir}${pathName}`, (error, data) => {
        if (!error) {
            response.writeHead(200);
            response.end(data);
        } else {
            response.writeHead(404);
            response.end('404 - File Not Found');
        }
    });
});

const getContentType = (pathName) => {
    let contentType = 'application/octet-stream';

    for (var key in mimeTypes) {
        if (mimeTypes.hasOwnProperty(key)) {
            if (pathName.indexOf(key) > -1) {
                contentType = mimeTypes[key];
            }
        }
    }

    return contentType;
};

server.init = (port = 3000, host = '127.0.0.1') => {
    httpServer.listen(port, () => {
        console.log(`Server is running at http://${host}:${port}`);
    });
};

server.init();
```

Practical 5

Counter using ReactJS

```
// App.js
import React, { useState } from 'react';

function App() {
    const [count, setCount] = useState(0);

    return (
        <div style={{textAlign: "center"}}>
            <h1>Counter</h1>
            <h2>{count}</h2>

            <button onClick={() => setCount(count + 1)}>
                Increment
            </button>

            <button onClick={() => setCount(count - 1)}>
                Decrement
            </button>

            <button onClick={() => setCount(0)}>
                Reset
            </button>
        </div>
    );
}

export default App;
```

Practical 6

Simple Login Form using React JS

```
// App.js
import './App.css';
import React, { useState } from "react";

function App() {
    const [errorMessages, setErrorMessages] = useState({});
    const [isSubmitted, setIsSubmitted] = useState(false);

    const database = [
        { username: "user1", password: "pass1" },
        { username: "user2", password: "pass2" }
    ];

    const errors = {
        uname: "invalid username",
        pass: "invalid password"
    };

    const handleSubmit = (event) => {
        event.preventDefault();

        var { uname, pass } = document.forms[0];

        const userData = database.find(
            (user) => user.username === uname.value
        );

        if (userData) {
            if (userData.password !== pass.value) {
                setErrorMessages({
                    name: "pass",
                    message: errors.pass
                });
            } else {
                setIsSubmitted(true);
            }
        } else {
            setErrorMessages({
                name: "uname",
                message: errors.uname
            });
        }
    };

    const renderErrorMessage = (name) =>
        name === errorMessages.name && (
            <div className="error">
                {errorMessages.message}
            </div>
        );

    const renderForm = (
        <div className="form">
            <form onSubmit={handleSubmit}>

                <div className="input-container">
                    <label>Username </label>
                    <input type="text" name="uname" required />
                    {renderErrorMessage("uname")}
                </div>

                <div className="input-container">
                    <label>Password </label>
                    <input type="password" name="pass" required />
                    {renderErrorMessage("pass")}
                </div>

                <div className="button-container">
                    <input type="submit" />
                </div>

            </form>
        </div>
    );

    return (
        <div className="app">
```

```
            <div className="login-form">
                <div className="title">Sign In</div>
                {isSubmitted
                    ? <div>User is successfully logged in</div>
                    : renderForm}
            </div>
        </div>
    );
}

export default App;

// App.css
.app {
    font-family: sans-serif;
    display: flex;
    align-items: center;
    justify-content: center;
    flex-direction: column;
    gap: 20px;
    height: 100vh;
}

input[type="text"],
input[type="password"] {
    height: 25px;
}

input[type="submit"] {
    margin-top: 10px;
    cursor: pointer;
    font-size: 15px;
    padding: 10px 20px;
}

.button-container {
    display: flex;
    justify-content: center;
}

.login-form {
    background-color: white;
    padding: 2rem;
    box-shadow: 0 4px 8px 0 rgba(0,0,0,0.2);
}

.error {
    color: red;
    font-size: 12px;
}

.title {
    font-size: 25px;
    margin-bottom: 20px;
}

.input-container {
    display: flex;
    flex-direction: column;
    gap: 8px;
    margin: 10px;
}
```

Practical 7

Chat Module using HTML, CSS and JavaScript

```
<!DOCTYPE html>
<html>
<head>
<title>Chat Module</title>

<style>
.container {
    border: 2px solid #dedede;
    background-color: #f1f1f1;
    border-radius: 5px;
    padding: 10px;
    margin: 10px 0;
}

.darker {
    border-color: #ccc;
    background-color: #ddd;
}

.container::after {
    content: "";
    clear: both;
    display: table;
}

.container img {
    float: left;
    max-width: 60px;
    width: 100%;
    margin-right: 20px;
    border-radius: 50%;
}

.container img.right {
    float: right;
    margin-left: 20px;
    margin-right: 0;
}

.time-right {
    float: right;
    color: #aaa;
}

.time-left {
    float: left;
    color: #999;
}
</style>
</head>

<body>

<div class="container">
    <img src="/w3images/bandmember.jpg" alt="Avatar">
    <p>Hello. How are you today?</p>
    <span class="time-right">11:00</span>
</div>

<div class="container darker">
    <img src="/w3images/avatar_g2.jpg" alt="Avatar" class="right">
    <p>Hey! I'm fine. Thanks for asking!</p>
    <span class="time-left">11:01</span>
</div>

<div class="container">
    <img src="/w3images/bandmember.jpg" alt="Avatar">
    <p>Sweet! So, what do you wanna do today?</p>
    <span class="time-right">11:02</span>
</div>

<div class="container darker">
    <img src="/w3images/avatar_g2.jpg" alt="Avatar" class="right">
    <p>Nah, I dunno. Play soccer.. or learn more coding perhaps?</p>
    <span class="time-left">11:05</span>
</div>
</body>
```

```
</html>
```